



Tecnológico de Monterrey

Escuela de Ingeniería y Ciencias

Campus Santa Fe

Analysis and Design of Advanced Algorithms

TC2038.622

Teacher: Lizbeth Peralta Malvárez

“Kruskal in C++”

Group: 601

Team:

Daniela Angulo Solís
Jonathan Uriel Anzures Garcia
Josue Gomez Arteaga

A01028153
A01277273
A01787212

Due date:

September 4th, 2026

Kruskal in C++

Daniela's reflection:

While working on the code and comparing it with Visual Go, I was able to understand the Kruskal algorithm much better. One thing I realized is that the order in which edges with the same weight are processed does not really matter. What is most important is finding the edges with the lowest weights and selecting them in a way that results in a low total weight.

Comparing our output against VisualGo step by step was especially useful for catching logic errors early. For example, verifying that the exact same edges were accepted and rejected at each step, not just that the totals matched by coincidence.

Josue's reflection:

Writing the union-find logic by hand helped me understand why more advanced implementations are useful. Our way of updating every node in a group works well for small graphs like this one, but it could become much slower when working with larger graphs with thousands of nodes. Every time two groups merge, we walk through the entire map to relabel the nodes that changed groups, which means the cost grows with the number of nodes already processed instead of staying close to constant.

A real Union-Find structure avoids this by using two tricks: union by rank/size, which always attaches the smaller tree to the larger one instead of picking a side arbitrarily, and path compression, which flattens the tree every time we look up a node's root so future queries are almost instant.

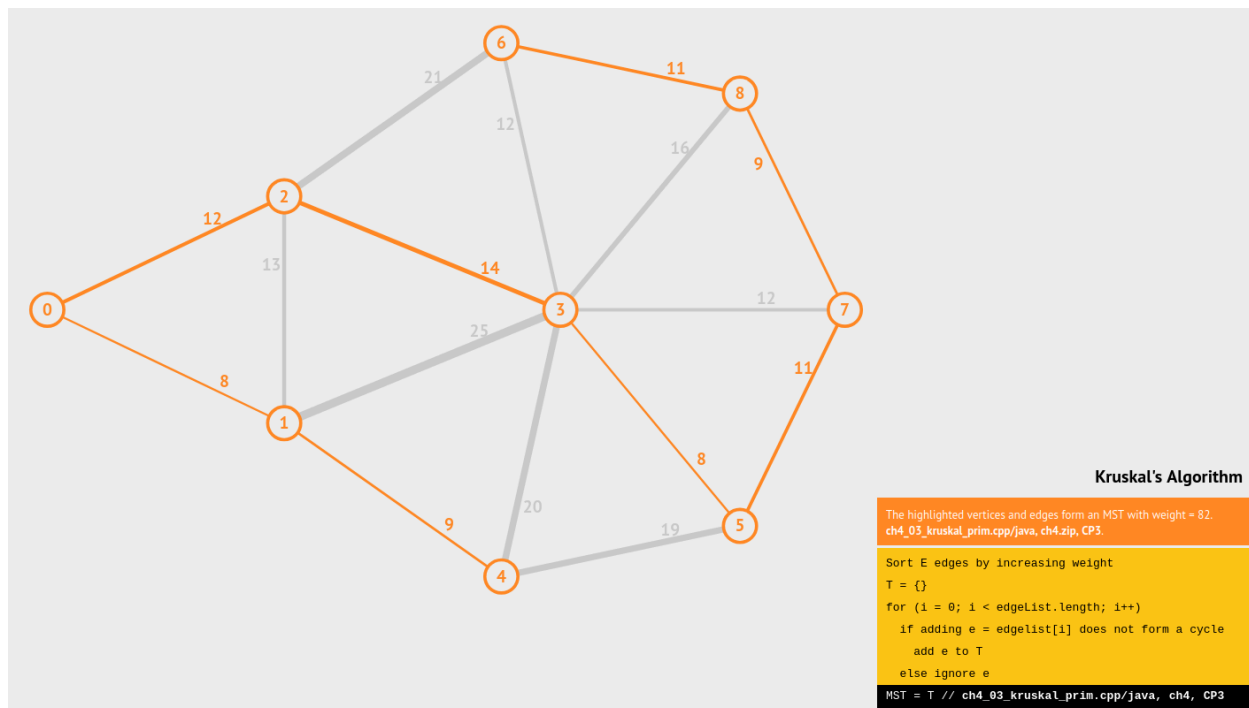
Uriel's reflection:

Writing the pseudocode for Kruskal's algorithm helped me understand better how the algorithm works step by step instead of only focusing on the final result. I understood that the main idea is to sort the edges by their weight and then check if adding each edge creates a cycle. If it does not create one, the edge is added to the minimum spanning tree.

Using VisualGo to compare our implementation was also useful because it allowed us to check if our logic was following the same process as the algorithm. Comparing the selected and rejected edges helped confirm that our pseudocode and C++ implementation were working correctly, even when some edges with the same weight were processed in a different order.

Comparison between our idea and Visual Go

Our C++ program was tested using the graph shown in the VisualGo simulation. Both implementations selected the exact same 8 edges: (0,1,8), (3,5,8), (1,4,9), (7,8,9), (5,7,11), (6,8,11), (0,2,12), and (2,3,14), producing a total weight of 82 in both cases. The edges (3,6,12), (3,7,12), and (1,2,13) were correctly rejected by both implementations for creating a cycle. The only difference observed was the internal order in which tied-weight edges (8-8, 9-9, 11-11) were processed, which does not affect the final result, since Kruskal's algorithm guarantees the same total weight regardless of tie-breaking order. This confirms that our C++ implementation produces a correct minimum spanning tree.



Img 1. Graph visualization in Visual Go (orange: selected edges, gray: skipped edges)

```
> ./kruskal
used      [0,1,8]
used      [3,5,8]
used      [1,4,9]
used      [7,8,9]
used      [5,7,11]
used      [8,6,11]
used      [0,2,12]
Skipped [3,7,12]  (already connected)
Skipped [3,6,12]  (already connected)
Skipped [1,2,13]  (already connected)
used      [2,3,14]
Skipped [3,8,16]  (already connected)
Skipped [4,5,19]  (already connected)
Skipped [3,4,20]  (already connected)
Skipped [2,6,21]  (already connected)
Skipped [1,3,25]  (already connected)

Selected edges:
[0,1,8],
[3,5,8],
[1,4,9],
[7,8,9],
[5,7,11],
[8,6,11],
[0,2,12],
[2,3,14]
Final weight: 82
```

Img 2. Our code output

Work division.

Josue and Daniela: Code and debugging

Uriel: Pseudocode

All: Reflection and description of the comparison